DOM REVISION PROJECTS

Official-Style Practical Project Specification

Scope: HTML + CSS + JavaScript DOM fundamentals through event delegation

Purpose

A structured hands-on project pack for revising DOM selection, content updates, class manipulation, element creation, traversal, events, event propagation, and event delegation. The projects are ordered from basic to integrated so each project reinforces the previous one.

Target learner

A learner who has completed DOM manipulation and event delegation theory and now wants implementation-heavy revision before moving to asynchronous JavaScript, APIs, and larger applications.

Working rule

Build the first version yourself. Use AI for explanations, debugging, edge cases, and design review rather than generating the entire implementation first.

Recommended stack

HTML5, CSS3, vanilla JavaScript (ES6+), browser DevTools. No frameworks, libraries, backend, or external API are required for Projects 1-6.

Project sequence	1. Counter App 2. Character Counter 3. Todo List 4. Todo List with Event Delegation 5. Tabs + Modal 6. Mini E-Commerce Cart
Difficulty progression	Beginner -> Intermediate -> Integrated DOM
Primary outcome	Ability to reason from user action -> event -> state change -> DOM update
Assessment	Each project includes functional requirements, edge cases, acceptance criteria, and a completion checklist.

1. General Project Standards

1.1 File structure

```
project-name/
index.html
style.css
script.js
```

1.2 Engineering expectations

• Use semantic HTML where practical: main, section, button, form, label, ul/ol, etc.

• Keep JavaScript in script.js unless a tiny experiment is intentionally inline.

• Prefer textContent for plain text updates; use innerHTML only when inserting trusted markup is genuinely required.

• Prefer classList for visual/state changes rather than repeatedly assigning large style strings.

• Use event delegation when the requirement explicitly calls for it or when child elements are created dynamically.

• Keep state variables distinct from DOM references. Example: cnt is state; counter is the DOM representation.

• Use clear variable and function names. Avoid unnecessary global variables.

• Test at least the normal path, boundary values, repeated clicks, and empty states.

1.3 Definition of done

Functional	Every stated feature works in a clean browser refresh.
Interaction	Buttons, inputs, keyboard interactions, and dynamically created elements behave as specified.
DOM	The implementation uses the requested DOM concepts rather than hard-coding the final screen.
Robustness	No console errors during normal use; boundary and empty states are handled.
Code quality	Repeated logic is extracted where appropriate; names are readable; no unnecessary duplication.

Project 1 - Counter App

Difficulty	Beginner
Primary goal	Revise DOM selection, textContent, addEventListener, classList, event delegation, state management, and a small UI update function.

2. User-facing features

- Increase counter by 1.
- Decrease counter by 1.
- Reset counter to 0.
- Display the current value in the UI.
- Positive values appear green, negative values appear red, and zero uses the default/black style.
- Use one delegated click listener on a suitable parent container.

3. Functional requirements

- Create a visible circular counter display.
- Provide three buttons: -, Reset, +.
- Maintain a numeric JavaScript state variable such as cnt.
- Select the required DOM nodes with querySelector/querySelectorAll as appropriate.
- Handle all button clicks through the parent listener.
- Identify the clicked action through event.target and/or closest.
- Create an updateCounter() function that synchronizes DOM text and visual state.
- Remove stale positive/negative classes before applying the current state.
- Use textContent for the numeric display.

4. DOM / JavaScript concepts under test

- querySelector
- textContent
- addEventListener
- event.target
- classList.add/remove
- event delegation
- state vs UI
- function extraction

5. Edge cases and test conditions

- Repeatedly click decrement below zero.
- Move from positive -> zero -> negative -> zero -> positive.
- Click reset repeatedly.
- Click rapidly many times.

6. Acceptance criteria

- + changes 0 to 1 and continues upward.
- - can create negative values.
- Reset always produces 0.
- Color/state always matches the current value.

• Only the intended buttons trigger state changes.
• No console errors.

7. Suggested implementation milestones

• Create semantic HTML shell and button classes.
• Select the parent and counter display.
• Create cnt state variable.
• Implement delegated click handling.
• Extract updateCounter().
• Add positive/negative classes and test state transitions.

```
function updateCounter() {
// 1. counter.textContent = cnt
// 2. remove old state classes
// 3. apply positive/negative state
}
```

8. Deliverables

• index.html, style.css, script.js
• A short README or notes file containing: approach, DOM concepts used, known limitations, and two improvements you would make next.
• A screenshot or short screen recording is optional but useful for self-review.

9. Self-review questions

• Why is the parent listener able to receive button clicks?
• Why should the DOM update be centralized?
• What happens if positive and negative classes are not removed?

10. Completion checklist

• [ ] Features implemented without copying a full solution
• [ ] Requirements tested one by one
• [ ] Edge cases tested
• [ ] Console checked for errors
• [ ] Code can be explained line by line
• [ ] At least one refactor performed after the first working version

Project 2 - Character & Word Counter

Difficulty	Beginner -> Intermediate
Primary goal	Revise input events, real-time DOM updates, string handling, and empty-state thinking.

2. User-facing features

• Textarea for user input.
• Live character count.
• Live word count.
• Live remaining-character count using a defined maximum.
• Clear button.
• Optional warning when the limit is near.

3. Functional requirements

• Use a textarea with a documented maximum length such as 200 characters.
• Update counts on every input event.
• Use textContent to render counters.
• Define word counting behavior explicitly: trim input; an empty trimmed string counts as 0 words.
• The clear button resets the textarea and all counters.
• If a maximum is used, prevent or clearly handle input beyond the limit.

4. DOM / JavaScript concepts under test

• querySelector
• input event
• textContent
• value
• trim
• split/filter or equivalent
• state synchronization

5. Edge cases and test conditions

• Completely empty input.
• Only spaces or newlines.
• Multiple spaces between words.
• Very long text near the limit.
• Pasting text.
• Clearing after typing.

6. Acceptance criteria

• Character count updates immediately while typing.
• Word count is stable for whitespace-only input and empty input.
• Clear returns all visible counters to their initial values.
• No page reload is required.
• No console errors.

7. Suggested implementation milestones

• Build markup and initial counters.
• Capture input event.
• Write a pure counting approach for characters/words.
• Update all counters in one render/update function.
• Test whitespace, paste, empty, and boundary inputs.

8. Deliverables

• index.html, style.css, script.js
• A short README or notes file containing: approach, DOM concepts used, known limitations, and two improvements you would make next.
• A screenshot or short screen recording is optional but useful for self-review.

9. Self-review questions

• How does an input event differ from a click event here?
• What should happen for input consisting only of spaces?
• Where should the counting logic live?

10. Completion checklist

• [ ] Features implemented without copying a full solution
• [ ] Requirements tested one by one
• [ ] Edge cases tested
• [ ] Console checked for errors
• [ ] Code can be explained line by line
• [ ] At least one refactor performed after the first working version

Project 3 - Todo List

Difficulty	Intermediate
Primary goal	Revise dynamic DOM creation, attributes, class manipulation, traversal, and multiple interactions.

2. User-facing features

• Add a todo.
• Mark a todo complete/incomplete.
• Delete a todo.
• Clear all todos.
• Show an empty-state message when there are no todos.
• Prevent accidental creation of blank tasks.

3. Functional requirements

• Use an input and Add button, or a small form.
• Create list items dynamically with createElement.
• Each todo must contain task text and controls for completion/deletion.
• Completed state must be represented using a CSS class.
• Delete must remove only the selected item.
• Clear All must remove all todos and restore the empty state.
• Do not hard-code individual todo items in HTML.
• Use traversal methods or data attributes where appropriate.

4. DOM / JavaScript concepts under test

• createElement
• append
• remove
• classList
• textContent
• DOM traversal
• form/input events

5. Edge cases and test conditions

• Empty input.
• Whitespace-only input.
• Long task text.
• Deleting the first/middle/last item.
• Completing and uncompleting an item repeatedly.
• Clearing an already empty list.

6. Acceptance criteria

• Every added task appears exactly once.
• Completion state is visually obvious and reversible.
• Delete removes the intended item only.
• Empty state appears only when the list is empty.

• Newly created todos behave consistently with existing ones.

7. Suggested implementation milestones

• Build input/list shell.

• Implement add using createElement.

• Add completion and delete controls.

• Implement clear-all and empty state.

• Refactor repeated DOM updates into helper functions.

8. Deliverables

• index.html, style.css, script.js

• A short README or notes file containing: approach, DOM concepts used, known limitations, and two improvements you would make next.

• A screenshot or short screen recording is optional but useful for self-review.

9. Self-review questions

• Why is createElement necessary for a dynamic list?

• How do you identify the todo that must be deleted?

• Why is classList preferable to repeated inline styles?

10. Completion checklist

• [ ] Features implemented without copying a full solution

• [ ] Requirements tested one by one

• [ ] Edge cases tested

• [ ] Console checked for errors

• [ ] Code can be explained line by line

• [ ] At least one refactor performed after the first working version

Project 4 - Todo List with Event Delegation

Difficulty	Intermediate+
Primary goal	Prove understanding of bubbling and delegation by handling dynamically created todo controls with a single parent listener.

2. User-facing features

• All Project 3 features.
• Exactly one click listener for todo-item actions on the list container.
• Support dynamically created todos without adding a listener to every button.
• Optional Edit action.

3. Functional requirements

• Attach one click listener to the todo-list container.
• Use event.target and/or closest to determine which control was activated.
• Use data attributes such as data-action and/or data-id for reliable identification.
• Do not add separate click listeners to every delete/complete button.
• New todo items added after page load must automatically work.
• Keep action handling easy to extend with another action.

4. DOM / JavaScript concepts under test

• event bubbling
• event delegation
• event.target
• closest
• dataset/data-*
• dynamic DOM

5. Edge cases and test conditions

• Click directly on a button.
• Click a nested span/icon inside a button if you add one.
• Add many todos dynamically.
• Delete an item and then act on neighboring items.
• Rapid add/delete sequences.

6. Acceptance criteria

• The parent listener handles all item actions.
• Dynamically created items require no new event listener registration.
• The correct item is modified or removed.
• Nested clickable markup does not break targeting.
• Console remains error-free.

7. Suggested implementation milestones

• Start from Project 3.
• Replace per-item action listeners with one parent listener.
• Add data-action and data-id.

• Use closest() where nested elements are possible.
• Verify newly created items work without extra listeners.

8. Deliverables

• index.html, style.css, script.js
• A short README or notes file containing: approach, DOM concepts used, known limitations, and two improvements you would make next.
• A screenshot or short screen recording is optional but useful for self-review.

9. Self-review questions

• Why does event delegation continue to work for newly created todos?
• What is the difference between target and currentTarget?
• When would closest() be safer than checking target.classList directly?

10. Completion checklist

• [ ] Features implemented without copying a full solution
• [ ] Requirements tested one by one
• [ ] Edge cases tested
• [ ] Console checked for errors
• [ ] Code can be explained line by line
• [ ] At least one refactor performed after the first working version

Project 5 - Tabs + Modal System

Difficulty	Intermediate+
Primary goal	Combine stateful UI patterns, classList, event propagation, keyboard events, and delegation.

2. User-facing features

• Three or more tabs.
• Only the active tab panel is visible.
• A modal opened by a button.
• Close modal with a close button.
• Close modal by clicking the overlay/background area.
• Close modal with Escape.

3. Functional requirements

• Tabs must have a clear active state.
• Tab-to-panel mapping must be explicit, preferably through data-* attributes.
• Use classes rather than inline style changes for visibility/state.
• Modal must have a hidden/default state and an open state.
• Overlay click should close only when the overlay itself is clicked, not when the modal content is clicked.
• Escape key should close the modal when it is open.
• Avoid attaching unnecessary listeners to repeated tab elements if delegation is suitable.

4. DOM / JavaScript concepts under test

• classList
• data-* / dataset
• event delegation
• event.target
• stopPropagation (only when justified)
• keydown
• preventDefault where relevant

5. Edge cases and test conditions

• Click the already active tab.
• Click rapidly between tabs.
• Click inside modal content.
• Click overlay outside modal content.
• Press Escape repeatedly.
• Open and close modal multiple times.

6. Acceptance criteria

• Exactly one tab/panel pair is active at a time.
• Correct panel is shown for every tab.
• Modal opens/closes reliably.
• Overlay behavior distinguishes background from content.
• Escape closes an open modal without throwing errors.

7. Suggested implementation milestones

• Build tabs and panels with data-target mapping.
• Implement active classes and panel visibility.
• Build modal open/close state.
• Add overlay and Escape handling.
• Test event propagation carefully.

8. Deliverables

• index.html, style.css, script.js
• A short README or notes file containing: approach, DOM concepts used, known limitations, and two improvements you would make next.
• A screenshot or short screen recording is optional but useful for self-review.

9. Self-review questions

• Why must overlay detection distinguish target from currentTarget?
• When is stopPropagation actually justified?
• How should tab state and modal state be represented in the DOM?

10. Completion checklist

• [ ] Features implemented without copying a full solution
• [ ] Requirements tested one by one
• [ ] Edge cases tested
• [ ] Console checked for errors
• [ ] Code can be explained line by line
• [ ] At least one refactor performed after the first working version

Project 6 - Mini E-Commerce Cart

Difficulty	Advanced revision project
Primary goal	Integrate nearly all DOM concepts learned so far into one stateful application without a framework.

2. User-facing features

• Render a product catalog from a JavaScript array.
• Add products to cart.
• Show cart items and quantities.
• Increase/decrease quantity.
• Remove item.
• Calculate subtotal/total.
• Search products.
• Filter products by a simple category.
• Show empty-cart state.
• Use delegation for dynamic product and cart actions.

3. Functional requirements

• Store product data in a JavaScript array of objects.
• Render product cards dynamically; do not hard-code every product card in HTML.
• Each product must have a stable identifier.
• Render cart from application state rather than manually editing unrelated DOM fragments.
• Use one delegated listener for product-card actions and/or one for cart actions.
• Quantity cannot go below 1 unless removal is the explicit action.
• Total must update after every cart mutation.
• Search and filter controls must re-render or update the visible product list correctly.
• Empty states must be explicit.
• Keep calculations separate from direct DOM operations where practical.

4. DOM / JavaScript concepts under test

• arrays/objects
• createElement/rendering
• dataset
• event delegation
• classList
• textContent
• filter/map/find/reduce
• state synchronization
• DOM traversal

5. Edge cases and test conditions

• Add the same product multiple times.
• Search for a nonexistent product.
• Filter to a category with zero matches.
• Remove the last cart item.

- Rapid quantity changes.
- Very long product names.
- Reset/refresh while no data persistence is implemented.

6. Acceptance criteria

- Product cards render from data.
- Adding the same product updates quantity rather than creating inconsistent duplicate state.
- Cart totals are mathematically correct.
- Every dynamic button works through delegation as specified.
- Search/filter and cart state do not corrupt one another.
- Empty catalog/cart states are handled cleanly.
- No console errors during normal use.

7. Suggested implementation milestones

- Define product data model.
- Render products.
- Create cart state and cart rendering.
- Implement delegated product/cart actions.
- Add search/filter.
- Add total calculation and edge-state handling.
- Refactor into small functions such as renderProducts, addToCart, updateCart, renderCart.

8. Deliverables

- index.html, style.css, script.js
- A short README or notes file containing: approach, DOM concepts used, known limitations, and two improvements you would make next.
- A screenshot or short screen recording is optional but useful for self-review.

9. Self-review questions

- What is application state versus DOM state?
- Why is stable product identity important?
- Which calculations can be tested without touching the DOM?

10. Completion checklist

- [ ] Features implemented without copying a full solution
- [ ] Requirements tested one by one
- [ ] Edge cases tested
- [ ] Console checked for errors
- [ ] Code can be explained line by line
- [ ] At least one refactor performed after the first working version

7. Final Revision Challenge - One-Day Assessment

After completing the six projects, rebuild one project from memory without looking at your old code. The objective is not speed; it is demonstrating that you can reconstruct the architecture from first principles.

Assessment scenario

Build a Todo List with Add, Complete, Delete, Clear All, Empty State, and event delegation. Add an Edit action only after the required features work.

Time-box

Suggested working time: 60-90 minutes. Spend the first 10 minutes designing the DOM and state model, the next 40-60 minutes implementing, and the final 10-20 minutes testing and refactoring.

Pass conditions

- No tutorial or complete AI-generated solution is copied.
- The parent list handles dynamic actions through delegation.
- The implementation distinguishes state from DOM representation.
- Empty/edge cases are handled.
- You can explain bubbling, target/currentTarget, classList, textContent, createElement, and closest without notes.

What to study next

After these projects feel comfortable, move to asynchronous JavaScript and browser data flow: promises, async/await, fetch, JSON, error handling, and API-driven DOM rendering. Continue practicing DSA separately rather than replacing it with frontend practice.

AI usage protocol for learning

- Ask for explanations and hints before asking for full code.
- When debugging, provide your code and explain what you expected versus what happened.
- After receiving a fix, retype the relevant part yourself and explain why it works.
- Use AI to generate test cases, edge cases, and code-review questions after you have an initial implementation.